

Token Averaging for Compute-Efficient Language Model Pretraining

Working draft: results are being added as experiments complete

Vardh*

July 2026

Abstract

The dominant cost of language model pretraining is proportional to the number of transformer positions processed: every raw token occupies one position, so seeing more data costs proportionally more compute. We study *token averaging*, a simple modification in which every k consecutive token embeddings are mean-pooled into a single transformer position before the first attention layer, so that a model trains on $k\times$ more raw text at the same, in fact slightly lower, compute. Because a k -averaged model natively scores one token per window, we make comparability part of the method: models are trained with random window offsets so that every position receives training signal, and all cross- k comparisons use an *offset-ensemble* evaluation that recovers a true full-sequence negative log-likelihood at approximately the cost of one baseline forward pass. On this strictly comparable metric, a 125M model with $2\times$ averaging improves perplexity by 3.4% over its baseline while spending $0.91\times$ the training FLOPs. Reading the loss-versus-compute curves at equal loss, the baseline needs $1.58\times$ more compute at 50M and $1.44\times$ at 125M to reach the averaged model’s loss. A controlled decomposition shows the gain comes from cheaper data throughput rather than from extended context: spending the same savings on longer raw context hurts on packed web text at these scales, while averaging matches a full-attention model at the same extended context for $1.26\times$ less training compute and half the inference KV cache. Finally, a sweep over pooling functions under a matched protocol identifies a learned rule combining content and position ($d_{\text{model}}+1+k$ parameters) as the best pooling at both compression ratios tested, improving on mean pooling by 0.037 nats at $k=2$ and 0.283 nats at $k=4$. [TODO: add 250M and 500M/1B endpoints; add full-position numbers at 50M/250M; tighten to venue length]

Contents

1	Introduction	2
2	Related Work	3
3	Method	4
3.1	Token averaging	4
3.2	Training objective	4
3.3	Random-offset training	5
3.4	Two deployment configurations: cheaper tokens vs. free context	5

*[TODO: author list, affiliations, contact]

3.5	FLOPs accounting	6
3.6	The evaluation problem and the offset-ensemble fix	6
3.7	Pooling functions	7
4	Experimental Setup	8
5	Results	9
5.1	Iso-FLOP comparison: averaging ahead at every scale	9
5.2	Full-position evaluation: the gain survives the strictly comparable metric	10
5.3	Decomposition: cheaper tokens, not free context	11
5.4	Pooling ablations: a learned content-plus-position rule wins at both compression ratios	12
6	Discussion and Limitations	14
7	Conclusion	15
A	FLOPs Derivation	15
B	Offset-Ensemble Evaluation: Details	16
C	Word-Aligned Windows: Algorithm and Caveats	16
D	Pooling Modules	17
E	Full Run Table	17
F	Training Instabilities and Excluded Runs	17
G	Hyperparameters	18

1 Introduction

Large language models are trained by next-token prediction over enormous corpora, and the arithmetic of that training is unforgiving: with standard architectures, each raw token of training data occupies exactly one position in the transformer, so the compute bill scales linearly with the amount of data seen, with a per-token cost that additionally grows with context length through the attention term. Compute-optimal scaling analyses (Hoffmann et al., 2022) show that parameters and data should be scaled in equal proportion, so half of every additional unit of compute is spent on data at every scale. This makes the *cost per token of data* a first-order constant of the training economy: any mechanism that lowers it without proportionally degrading what the model learns per token converts directly into lower loss at fixed compute, or equal loss at lower compute.

This paper studies the simplest such mechanism we know of: *token averaging*. Before the first transformer layer, we partition the input sequence into non-overlapping windows of k consecutive tokens and replace each window’s k embedding vectors by their mean. The transformer then runs on T/k positions instead of T , and is trained to predict the first token of the following window at each compressed position. Nothing else changes: no new parameters are introduced (the pooling is a fixed mean), and the architecture, tokenizer, optimizer, and data pipeline are identical to the baseline. Each raw token now costs roughly $1/k$ of the baseline’s compute, so at a fixed budget the model trains on $k\times$ more raw data; because the shorter transformer sequence also cuts the quadratic attention cost, the averaged model in fact finishes *below* the baseline’s FLOPs ($0.87\times$ at our 50M configuration, $0.91\times$ at 125M; Section 3.5). The hypothesis is that transformers can model mildly compressed input representations nearly as well as raw ones, in which case a k -averaged model can afford to train on $\sim k\times$ more raw data at the same or lower compute and end up strictly ahead.

Comparing such a model to a standard one requires care, and we build the required machinery into the method itself. A k -averaged model natively assigns probabilities to one token per window, so its training loss averages over a different set of conditionals than the baseline’s and the two native losses are not measurements of the same quantity. We therefore (i) train with a *random window offset* per step, so that every token position receives equal training signal (Section 3.3), and (ii) evaluate every cross- k comparison with an *offset-ensemble* protocol: the model is run k times per evaluation sequence with the window grid shifted by $o = 0, \dots, k-1$ tokens, so that every position beyond the first window is predicted exactly once from its full compressed prefix (Section 3.6). This recovers a genuine full-sequence negative log-likelihood at approximately the cost of a single baseline forward pass, and the same construction yields a rotating-offset decoding scheme for token-by-token generation (Appendix B).

Our main configuration, Config A, holds the raw sequence length at the baseline’s value and banks the compute saving: the transformer runs on $1024/k$ positions, each prediction still conditions on 1024 raw tokens of context, and the freed compute buys $k\times$ the training data. This is the configuration we scale. To identify where its gains come from, we run two further arms as ablations: Config B, which holds the transformer length fixed and spends the entire saving on $k\times$ longer raw context instead, and a standard model with a genuinely widened attention window, which controls for whether any context effect is specific to averaging (Section 3.4).

Findings. Our main experimental findings, on FineWeb with GPT-NeoX-style models trained from scratch, are as follows.

1. **Averaging wins at iso-FLOPs, on a strictly comparable metric.** Under offset-ensemble evaluation, with identical evaluation sequences and identical target positions (3,409,119 predictions each), the 125M $k=2$ model scores 4.141 nats/token versus 4.177 for the baseline:

3.4% lower perplexity at $0.91\times$ the training FLOPs. The model’s per-offset losses are essentially equal (4.145 at offset 0 vs. 4.138 at offset 1), as expected for a model trained with random offsets over packed data (Section 5.2). Reading the loss-versus-compute curves at equal loss, the baseline needs $1.58\times$ more compute at 50M and $1.44\times$ at 125M to reach the averaged model’s loss (Section 5.1). [TODO: add 250M endpoint and multiplier; add 50M/250M full-position numbers]

2. **The gain is data-per-FLOP, not context.** In the decomposition experiment at 50M, extended raw context hurts on packed FineWeb in every arm: widening a standard model’s attention window from 1024 to 2048 costs +0.214 nats at $1.26\times$ the compute, and spending the averaging saving on longer raw context (Config B) instead of more data (Config A) is worse at both $k=2$ and $k=4$. The benefit of averaging on this data is therefore data throughput per FLOP (Section 5.3).
3. **When raw context is the goal, averaging obtains it more cheaply than full attention.** The averaged and full-attention 2048-context arms finish at the same loss, with the averaged route spending $1.26\times$ less training compute and holding half the KV cache at inference (Section 5.3). [TODO: full-position endpoints for the two 2048-context arms]
4. **A learned pooling rule combining content and position is the best pooling at both compression ratios.** In a matched-protocol sweep over six pooling functions at 50M, a $(d_{\text{model}}+1+k)$ -parameter scorer with a per-slot positional bias beats every alternative at both $k=2$ (4.326 vs. 4.363 for mean) and $k=4$ (4.163 vs. 4.446 for mean). The sweep also isolates *why*: at $k=4$ a fixed recency weighting alone recovers most of the gap (4.265) while a content-only learned scorer recovers none of it (4.445), so positional structure within the window, not content, is the missing ingredient that mean pooling discards (Section 5.4).

Contributions. (1) A systematic, FLOP-controlled study of token averaging as a *permanent* operating point, not a warm-up phase, across three model scales, with training budgets set for iso-FLOP endpoint comparisons. (2) A comparability toolkit for patch-style models: random-offset training, the offset-ensemble evaluation protocol that makes them comparable to token-level baselines on full-sequence likelihood, and a rotating-offset decoding design for generation. (3) A decomposition showing that extended raw context hurts on packed web text at these scales, so the benefit of averaging is cheaper data throughput; and that when context is wanted, averaging is the cheaper route to it. (4) A matched-protocol pooling sweep identifying content-plus-position learned pooling as the best rule at both compression ratios, and isolating the positional prior as the ingredient that plain mean pooling lacks.

2 Related Work

[TODO: This section is deliberately deferred. Anchor points to cover when we write it:]

- **Patch-level training.** Shao et al. (2024; ICLR’25), “Patch-Level Training for Large Language Models”: the closest prior work. It averages K token embeddings into patches and predicts the next patch, but uses patch-level training only as a *warm-up* followed by token-level fine-tuning, at 370M–2.7B scale. Our deltas: patch-level as the final operating point (made comparable by the offset-ensemble evaluation), FLOP-controlled scaling-law treatment rather than single budget points, and the context-extension decomposition.

- **Hierarchical / byte-level patching.** MegaByte, Byte Latent Transformer (dynamic patch boundaries), Block Transformer, Funnel-Transformer, Hourglass. These change the architecture to compress and re-expand; we deliberately keep the architecture untouched.
- **Multi-token prediction.** Gloeckle et al. 2024; DeepSeek-V3’s MTP objective. Related through the phased multi-token variant we briefly explore, but MTP predicts multiple targets from uncompressed inputs; we compress inputs.
- **Context compression.** Gist tokens, AutoCompressor, ICAE: these compress *prompts* at inference; we compress *all* input during pretraining.
- **Scaling laws.** Hoffmann et al. 2022 (Chinchilla) for the compute-optimal framing and the $D^* \approx 20N$ heuristic we use to set budgets; Kaplan et al. 2020.
- **Tokenizer granularity.** Work on tokenizer vocabulary size and bits-per-byte comparisons, relevant to the coarser-tokenizer control we plan (Section 6).

3 Method

3.1 Token averaging

Let $t_1, \dots, t_T$ be a sequence of raw tokens and $e_i \in \mathbb{R}^{d_{\text{model}}}$ their (learned) input embeddings. For a fixed averaging factor k , we partition the sequence into $n = \lfloor T/k \rfloor$ non-overlapping windows of k consecutive tokens and compress each window to a single vector by mean pooling:

$$\bar{e}_j = \frac{1}{k} \sum_{i=(j-1)k+1}^{jk} e_i, \quad j = 1, \dots, n. \quad (1)$$

The transformer body (attention layers, feed-forward layers, RoPE, final norm) runs unchanged on the compressed sequence $\bar{e}_1, \dots, \bar{e}_n$, producing n output states, and the output head produces logits over the vocabulary at each compressed position. With $k=1$ the method reduces exactly to the standard language model. Mean pooling introduces *no new parameters*; Section 3.7 describes the alternative pooling functions we ablate, the largest of which adds 517 parameters.

Positional information deserves a comment. Rotary position embeddings are applied inside the transformer body to the *compressed* positions $1, \dots, n$, so adjacent transformer positions are always k raw tokens apart. The geometry of the compressed sequence is therefore identical regardless of where the window grid is anchored in the raw text, a fact that becomes important for the offset analysis in Section 3.6.

3.2 Training objective

At compressed position j (windows and positions 1-indexed throughout) the model is trained to predict the *first token of the next window*: the target at position j is t_{jk+1} . Concretely, for $k=2$ on eight raw tokens:

raw tokens	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
averaged input	$\bar{e}_1 = \frac{e_1+e_2}{2}$		$\bar{e}_2 = \frac{e_3+e_4}{2}$		$\bar{e}_3 = \frac{e_5+e_6}{2}$		$\bar{e}_4 = \frac{e_7+e_8}{2}$	
target	$\rightarrow t_3$		$\rightarrow t_5$		$\rightarrow t_7$		(none)	

The loss is the standard cross-entropy averaged over the $n - 1$ predicted positions. Note two properties that drive much of the paper. First, the model predicts only the tokens $t_{k+1}, t_{2k+1}, t_{3k+1}, \dots$, one per window; the conditionals for all other positions are simply absent from the model as trained, which is the root of the evaluation comparability problem (Section 3.6). Second, each prediction conditions on a *lossy* summary of the past: the most recent window contributes only through its mean, so fine within-window order information is discarded. How much this costs, and whether smarter pooling recovers it, is the subject of Section 5.4.

We also experimented with a phased “token superposition” variant in which, for the first fraction of training, each compressed position predicts *all* k tokens of the next window as a bag (multi-hot cross-entropy through the single LM head), before reverting to standard single-token prediction. We treat it as an auxiliary experiment rather than part of the main method and report it only in the appendix. [TODO: decide whether phased results are in or out of the paper]

3.3 Random-offset training

With the window grid anchored at the start of the sequence, the model would only ever be asked to predict the raw positions $jk + 1$, i.e. $t_{k+1}, t_{2k+1}, \dots$ of each training sequence. Sequence packing already randomizes the grid *relative to the underlying text* (Section 3.6), but we remove the asymmetry at the source: at each training step we sample an offset $o \sim \text{Uniform}\{0, \dots, k-1\}$ per batch, drop the first o tokens of the batch, and proceed as usual. Over many steps every residue class of positions receives equal training signal. At evaluation time the offset is fixed to 0 (or swept, for the offset-ensemble metric). The change costs at most one window per sequence and touches nothing else.

3.4 Two deployment configurations: cheaper tokens vs. free context

At a fixed FLOP budget, the compute freed by averaging can be spent in two distinct ways. Config A, which spends it on data, is the paper’s method and the configuration we scale; Config B, which spends it on context, serves as an ablation that isolates where Config A’s gains come from. Throughout, `seq_len` denotes the number of *raw* tokens per training sequence and $L = \text{seq_len}/k$ the number of positions the transformer actually processes.

Config A (cheaper tokens; the paper’s main configuration). The raw `seq_len` stays at the baseline’s value (1024). The transformer length shrinks to $L = 1024/k$, each raw token costs roughly $1/k$ of baseline, and the model trains on $k \times$ the raw tokens for the same endpoint FLOPs. The raw context available to any prediction is unchanged (1024 tokens); only its representation is compressed.

Config B (extended context; ablation). Scale `seq_len` up to $k \times 1024$ so that $L = 1024$ is *identical* to the baseline. Now each sequence embeds $k \times$ the raw context into the same transformer computation, and per raw token the cost is again $1/k$ of baseline, giving iso-FLOPs at $k \times$ tokens by construction.

Both configurations use identical models, data, and optimization; they differ only in where the savings go. To determine whether any Config B effect is specific to averaging, we add a control arm: a standard $k=1$ model with a genuinely widened attention window (`seq_len` = L = 2048), which obtains the same raw context the traditional way at $1.26 \times$ the baseline’s per-token cost.

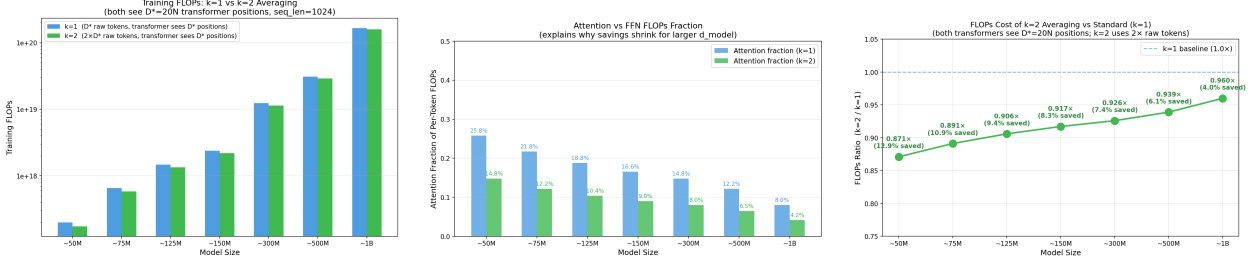


Figure 1: FLOPs structure across model widths. **Left:** absolute training FLOPs per raw token. **Middle:** the attention share of per-position compute shrinks with d_{model} . **Right:** the per-token FLOPs ratio of $k=2$ averaging to baseline at the iso-data multiple across widths. The dominant, width-independent effect of averaging is the $k\times$ data throughput; the attention saving adds to it and is largest at small widths.

3.5 FLOPs accounting

We recompute all FLOPs from first principles rather than trusting logged values. For a transformer with hidden size d , n_{layers} layers, and transformer sequence length L , our architecture (attention with Q, K, V , output projections; SwiGLU FFN with expansion factor 2.5) costs, per layer and per position, $23d^2 + 4Ld$ multiply-accumulate FLOPs forward: $8d^2$ for the four attention projections, $15d^2$ for the SwiGLU FFN (see Appendix A for the exact derivation), and $4Ld$ for the attention score and value aggregation terms. With the usual $3\times$ multiplier for forward-plus-backward, a training run that consumes D raw tokens at raw sequence length seq_len costs

$$\text{FLOPs}(D) = \frac{D}{\text{seq_len}} \cdot n_{\text{layers}} \cdot L \cdot 3(23d^2 + 4Ld), \quad L = \text{seq_len}/k. \quad (2)$$

Equation (2) makes the economics of Config A transparent, and it is worth separating two comparisons that are easy to blur. *At a fixed raw-token count D* , averaging processes D/k positions instead of D , so the linear-term cost ($23d^2$ per position) per raw token falls by a factor of k , and the attention term falls by k^2 (a factor k from fewer positions and another from $L \rightarrow L/k$). *At the iso-FLOP deployment*, the averaged run converts that saving into kD raw tokens of training data, which returns the linear-term total to the baseline’s while the attention term stays lower, since its per-position cost depends on the shorter L ; this is why the averaged arms finish below the baseline’s endpoint FLOPs ($0.87\times$ at 50M, $0.91\times$ at 125M) even after consuming $k\times$ the data. The primary quantity token averaging buys is therefore *data throughput*: $k\times$ more raw data flows through the same transformer budget, with the attention saving on top, and the empirical question is what that throughput is worth in loss.

3.6 The evaluation problem and the offset-ensemble fix

The problem. A language model’s defining scalar is the full-sequence log-likelihood $\sum_i \log p(t_i | t_{<i})$. The $k=1$ baseline’s eval loss is the mean of all $T-1$ such terms per sequence. The k -averaged model’s eval loss, as trained, is the mean over only the $\sim T/k$ positions it predicts; the remaining conditionals do not exist in the model. The two losses are therefore averages over *different sets of terms*, and no amount of extra evaluation data fixes this: matching the *count* of predicted tokens by evaluating on $k\times$ the batches matches sample sizes, not tasks. An additional casualty is generation: a model that predicts one token per window cannot decode token-by-token as trained.

The fix. For evaluation, run the model k times over each sequence, shifting the window-grid anchor by $o = 0, 1, \dots, k-1$ raw tokens. At offset o , windows cover tokens $(o+1..o+k), (o+k+1..o+2k), \dots$ and the model’s targets are the raw positions $o + jk + 1$ for $j = 1, 2, \dots$, i.e. $t_{o+k+1}, t_{o+2k+1}, \dots$. Across the k offsets these target sets are disjoint, and their union is every position $> k$ (all residue classes are covered), each predicted exactly once conditioned on its full compressed prefix. This yields a genuine full-sequence NLL over all scored positions, directly comparable to the baseline’s on the same positions. The cost is k forward passes over sequences of length $L = T/k$, approximately *one* baseline forward pass in total, since per-pass cost scales with L . For an airtight comparison we additionally score the baseline restricted to the identical set of target positions, eliminating even the residual composition difference from the first k tokens. The same construction yields a decoding scheme for generation: rotate the offset during decoding so that a window boundary always ends at the current position, maintaining k rotating KV caches whose amortized cost matches one baseline decoder (Appendix B).

Why offset generalization comes cheap. Beyond the explicit random-offset training of Section 3.3, two structural facts make offset robustness the expected behaviour rather than an extra ability the model must learn. First, training sequences are produced by packing a continuous token stream into fixed-length chunks whose boundaries fall at arbitrary points of the underlying documents, so the window grid’s alignment *relative to the text* is uniformly random over training regardless of the offset used. Second, RoPE operates on compressed positions (Section 3.1), so every offset induces identical positional geometry. Random-offset training formalizes what packing already does; consistently, the measured per-offset losses are essentially equal (Section 5.2).

3.7 Pooling functions

Mean pooling (Eq. 1) is the simplest possible aggregation, and everything upstream of the transformer flows through it. We therefore ablate the pooling function while holding all else fixed. All variants below compress k (or a window of) embeddings to one vector and leave the rest of the pipeline untouched.

Mean (default). Equal weights $1/k$. No parameters.

Fixed weighted (exponential). $\bar{e} = \sum_i w_i e_i$ with $w_i \propto \exp(\alpha i)$, α set so the last-to-first weight ratio is 20, then L_1 -normalized. A pure recency prior: the most recent token in a window dominates. No learned parameters. (We also implemented linear, Gaussian, and triangular schemes; the exponential scheme is the representative we trained.)

Learnable (content-based). A single shared linear scorer $s_i = w^\top e_i + b$ produces one scalar per token; weights are $\alpha = \text{softmax}(s_{1..k})$ within each window and $\bar{e} = \sum_i \alpha_i e_i$. This adds $d_{\text{model}} + 1 = 513$ parameters, zero-initialized so training starts exactly at mean pooling. The scorer sees each embedding independently, *before* any positional information exists in the network: it is content-aware but position-blind, a property that turns out to matter (Section 5.4).

Learnable (content + position). The same scorer plus a learned per-slot bias: $s_i = w^\top e_i + b + p_i$ with $p \in \mathbb{R}^k$ trained jointly ($d_{\text{model}} + 1 + k$ parameters). We initialize p at uniform for $k=2$ and at the exponential recency profile for $k=4$, i.e. at the best fixed rule found for each compression ratio, so the model starts from the strongest known prior and can adapt both terms end-to-end.

Overlapping windows. Average windows of $w=4$ tokens with stride $s=2$: the same $2\times$ compression as $k=2$, but adjacent windows share tokens, so no token sits on a hard boundary. Targets are the first token beyond each window.

Word-aligned windows. Windows of variable size ($\geq k$, force-closed at $2k$) that end only at word boundaries, detected via the tokenizer’s whitespace-prefix convention, so no window blends two words. Realized compression on FineWeb is $\sim 2.1\times$ at $k=2$ (slightly above nominal), and the predicted positions are word starts; both caveats affect comparability and are handled explicitly (Appendix C).

4 Experimental Setup

Architecture. All models are GPT-NeoX-style decoder-only transformers built from the OLM implementation: pre-norm blocks, rotary position embeddings, SwiGLU FFN with expansion factor 2.5, and a padded vocabulary of 50,304 (tokenizer vocabulary 50,257). Table 1 lists the three scales. Input and output embeddings are tied at all scales.

Table 1: Model configurations. “Raw tokens” are the training budgets for the $k=1$ / $k=2$ arms, set so both arms end at approximately equal total training FLOPs (Eq. 2); the $k=1$ budgets follow the Chinchilla $D^* \approx 20N$ heuristic. All main runs use raw `seq_len` = 1024 (Config A).

Scale	Params	d_{model}	Heads	Layers	LR	Raw tokens ($k=1$ / $k=2$)
50M	50.9M	512	8	8	2.0×10^{-4}	1B / 2B
125M	123.5M	768	12	12	1.6×10^{-4}	2.5B / 5B
250M	252.8M	1024	16	16	1.4×10^{-4}	5B / 10B

Data and tokenization. All models train from scratch on FineWeb (`sample-10BT`), tokenized with the EleutherAI Pythia-70m (GPT-NeoX) BPE tokenizer. The token stream is packed into fixed-length sequences without document-boundary masking; this is standard practice, but consequential for the context experiments (Section 5.3). Evaluation uses a held-out split of the same distribution, identical across all arms of a comparison.

Optimization. AdamW with a cosine learning-rate schedule: 2,000 warmup steps, decay to 10% of peak over exactly the run’s total step count. Peak learning rates follow Table 1. Mixed-precision (bfloat16) training on NVIDIA GPUs via PyTorch DDP; gradient checkpointing at 250M. Batch sizes and step counts per run are tabulated in Appendix E, and we return to the optimizer-step confound they induce in Section 5.4.

Budgets and iso-FLOP design. For each scale the $k=1$ arm trains for approximately $20N$ raw tokens and the $k=2$ arm for twice that, which by Eq. (2) places the two endpoints at approximately equal training FLOPs (exactly equal in the linear terms; the $k=2$ arm actually ends *below* the baseline’s FLOPs because its attention term is smaller: $0.87\times$ at 50M, $0.91\times$ at 125M). We report both the vertical reading (equal compute, compare loss) and the horizontal reading (equal loss, compare compute) of every curve, because late-training loss curves are so flat that small vertical gaps correspond to large horizontal ones.

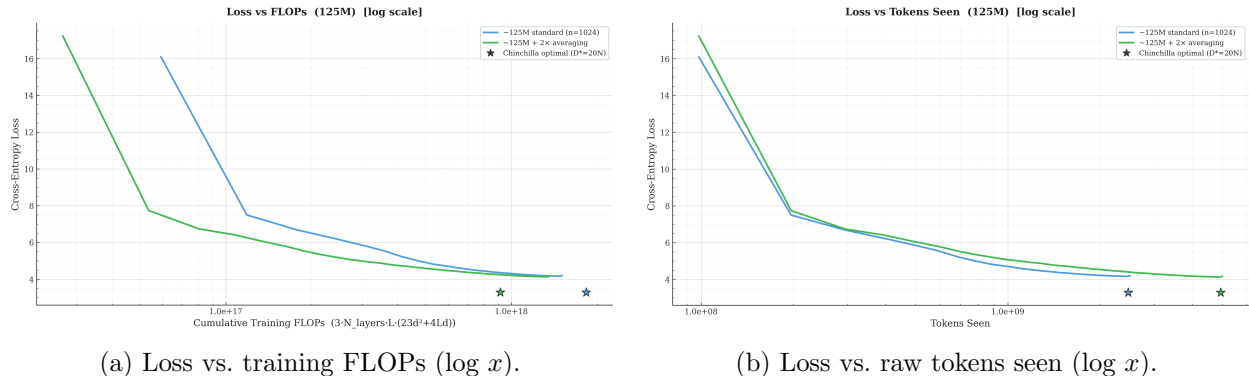


Figure 2: 125M comparison, native eval losses. On the FLOPs axis (left) the $k=2$ curve sits below the baseline curve along the whole trajectory at equal FLOPs; the endpoint comparison on the full-position metric is in Section 5.2. On the raw-token axis (right) the averaged model learns slightly *less* per token (averaging is lossy), but each of its tokens costs roughly half the compute. Stars mark Chinchilla-optimal reference points ($D^*=20N$).

Metrics. Two metrics appear in this paper, with a fixed division of labor. “Native eval loss” is cross-entropy in nats per predicted token on held-out data at the model’s native prediction positions (offset 0); it is the quantity logged during training, and we use it for training curves and for comparing runs with the same k and window scheme. The “full-position NLL” is the offset-ensemble metric of Section 3.6; it scores every position of the identical evaluation sequences and is the metric for every comparison across k . All final checkpoints are evaluated under both. [TODO: fill in full-position numbers for the 50M and 250M pairs and the context arms as runs complete]

5 Results

5.1 Iso-FLOP comparison: averaging ahead at every scale

This subsection reports the training trajectories and endpoint losses of the main Config A pairs; the corresponding full-position comparison, the metric for all cross- k conclusions, is given in Section 5.2.

Figure 2 shows the 125M pair; Table 2 summarizes endpoints at all scales. Three observations.

Table 2: Main iso-FLOP comparison (Config A, raw `seq_len` 1024). Losses are final native eval CE (nats/token) at each model’s own prediction positions; the cross- k endpoint comparison on the full-position metric is in Section 5.2. Train FLOPs are computed from tokens seen via Eq. (2) in all rows. The equal-loss saving is read horizontally off the loss–FLOPs curves. [TODO: replace 250M rows with final endpoints; add 500M/1B]

Scale	Arm	Raw tokens	Train FLOPs	Final eval	vs. base	Saving
50M	$k=1$	1.0B	1.95×10^{17}	4.437	–	
50M	$k=2$	2.0B	1.70×10^{17}	4.363	0.87×	1.58×
125M	$k=1$	2.5B	1.50×10^{18}	4.204	–	
125M	$k=2$	5.0B	1.36×10^{18}	4.171	0.91×	1.44×
250M (prelim.)	$k=1$	4.18B / 5B	5.68×10^{18}	3.458	–	
250M (prelim.)	$k=2$	8.36B / 10B	5.26×10^{18}	3.438	–	[TODO:]

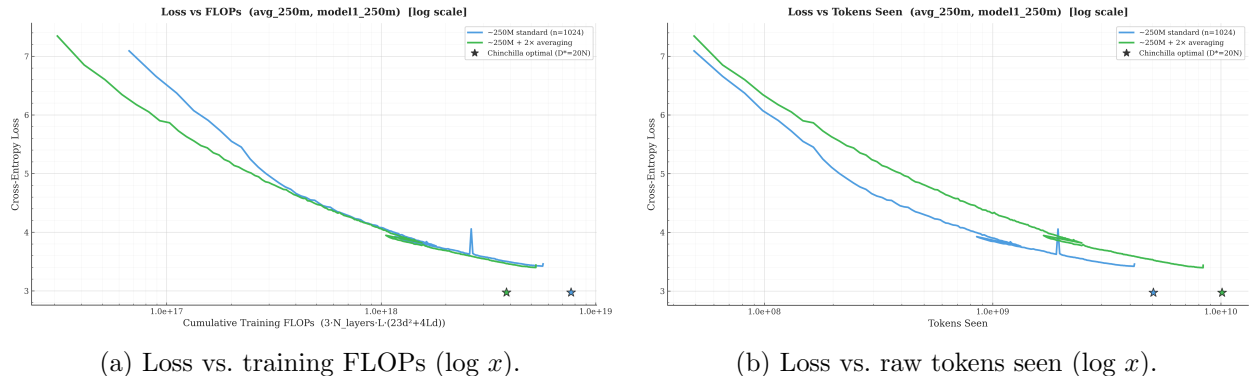


Figure 3: 250M comparison on native eval losses (preliminary; both arms short of their target budgets: $k=1$ at 4.18B of 5B raw tokens, $k=2$ at 8.36B of 10B). The qualitative picture matches the smaller scales.

First, the averaged model ends below the baseline’s loss at every scale measured, while spending fewer FLOPs at its endpoint ($0.87\times$ at 50M, $0.91\times$ at 125M). The vertical gaps are modest (0.074 nats at 50M, 0.033 nats at 125M), but the horizontal reading is the operationally meaningful one: at 125M the baseline requires its full 1.50×10^{18} FLOPs to reach loss 4.204, whereas the averaged model crosses that loss at 1.05×10^{18} , a $1.44\times$ compute saving; the same construction at 50M gives $1.58\times$. Late-training loss curves are extremely flat, so small vertical differences translate into large horizontal ones, which is why we report both readings.

Second, the equal-loss multiplier is large at both scales measured ($1.58\times$ at 50M, $1.44\times$ at 125M). How the multiplier behaves as scale grows is the central quantitative question of this study, and the scaling arm of the paper answers it with a Chinchilla-style fit of loss-versus-compute per family, with the efficiency multiplier as a function of scale. [TODO: add 250M multiplier (preliminary endpoints in Table 2; loss curves in Figure 3) and the 500M/1B points; fit $L(C)$ per family; add crossover analysis]

Third, the mechanism is visible in the raw-token view (Figure 2b): per raw token the averaged model learns *slightly less* than the baseline, since compression is genuinely lossy, but it processes each token at roughly half the compute, and the trade nets out positive. Token averaging does not make the model smarter per token; it makes tokens cheap enough that seeing twice as many wins.

5.2 Full-position evaluation: the gain survives the strictly comparable metric

The curves above show native eval losses, which Section 3.6 argued are not an apples-to-apples quantity across k . Table 3 therefore scores the final checkpoints with the offset-ensemble protocol: identical held-out sequences, identical target positions (3,409,119 predictions per model at 125M).

At 125M the averaged model is better by 0.035 nats, a 3.4% perplexity improvement, on a metric that scores every position exactly once from its full compressed prefix, while having spent $0.91\times$ the training FLOPs. The evaluation-comparability objection is answered on the merits, not argued around.

The per-offset decomposition confirms the structural argument of Section 3.6: offset 0 scores 4.145 and offset 1 scores 4.138, essentially equal. No offset is privileged, which is what random-offset training over packed data should produce, and it is the property that makes the ensemble metric a faithful full-sequence likelihood rather than a mixture of one in-distribution and $k-1$ out-of-distribution passes. The same property is what rotating-offset generation relies on (Appendix B).

Table 3: Offset-ensemble full-position evaluation of the final Config A checkpoints. Within each scale, both rows average over the identical set of scored positions (every position $> k$ of each evaluation sequence); the baseline is restricted to exactly the position set covered by the $k=2$ ensemble. [TODO: fill 50M and 250M rows]

Scale	Model	Mean NLL (nats/token)	PPL	Coverage
125M	$k=1$ baseline (restricted)	4.177	65.1	all scored positions
125M	$k=2$, 2-offset ensemble	4.141	62.9	all scored positions
50M	$k=1$ baseline (restricted)	[TODO:]		
50M	$k=2$, 2-offset ensemble	[TODO:]		
250M	$k=1$ baseline (restricted)	[TODO:]		
250M	$k=2$, 2-offset ensemble	[TODO:]		

Together, the pair (random-offset training, offset-ensemble evaluation) is what makes patch-style training viable as a *final* operating point rather than a warm-up.

5.3 Decomposition: cheaper tokens, not free context

Config A gains could in principle come from two entangled sources: more data per FLOP, or (in the Config B deployment) more raw context per prediction. The 50M decomposition experiment separates them. Four arms, all approximately iso-FLOPs except the deliberately more expensive full-attention control: the baseline ($L=1024$, 1024 raw context); Config B $k=2$ (`seq_len` 2048, $L=1024$, so $2\times$ raw context at identical transformer cost); Config B $k=4$ (4096 raw context); and a $k=1$ model with true 2048-token attention ($1.26\times$ baseline cost).

Because native losses are only comparable within a fixed k and window scheme, we state the evidence as within-method comparisons: each extended-context arm against the same- k arm without extended context.

Table 4: Context-extension experiment at 50M, native eval losses. The Δ column gives the within-method comparison: each extended-context arm against the same- k arm without extended context ($k=1$ full attention vs. the $k=1$ baseline; Config B $k=2$ vs. Config A $k=2$ at 4.363; Config B $k=4$ vs. Config A $k=4$ at 4.446). Losses in different k rows are not mutually comparable.

Arm	Raw ctx	L	Raw tokens	Train FLOPs	Final eval (Δ within method)
$k=1$ baseline	1024	1024	1.00B	1.95×10^{17}	4.437
$k=1$ full attention	2048	2048	1.00B	2.45×10^{17}	4.651 (+0.214)
Config B, $k=2$	2048	1024	2.00B	1.95×10^{17}	4.650 (+0.287 vs. Config A)
Config B, $k=4$	4096	1024	4.07B	1.99×10^{17}	5.174 (+0.728 vs. Config A)

Table 4 and Figure 4 give the result, which is consistent across every comparison and is one of the paper’s more useful negative findings. **Extending raw context does not pay for itself at this scale, within either method.** Within $k=1$, widening the attention window from 1024 to 2048 costs +0.214 nats while also costing $1.26\times$ the compute. Within $k=2$, spending the averaging saving on doubled raw context (Config B, 4.650) instead of more data (Config A, 4.363) costs +0.287 nats on the same prediction task. Within $k=4$ the corresponding penalty grows to +0.728 nats. The failure is a property of the data and scale, not of averaging: FineWeb documents are mostly

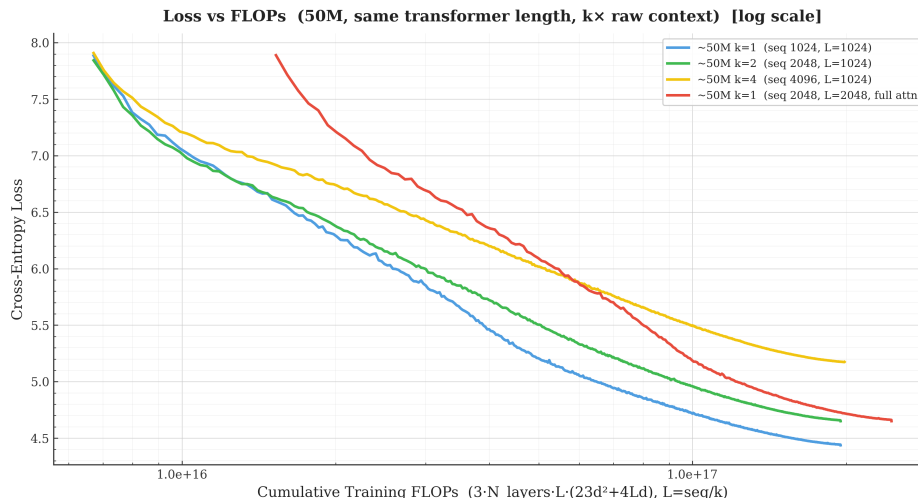


Figure 4: Context-extension experiment at 50M: native eval loss vs. training FLOPs ($\log x$). Curves of different k score different position sets and are overlaid for orientation; the valid comparisons are within-method (see text). The two 2048-row-context arms (green: $k=2$ averaging; red: full attention) end near the same native loss at very different cost; $4\times$ context (yellow) is worse still. Early-training loss spikes are clipped at 8.0 for readability.

shorter than 1024 tokens, and packed sequences cross document boundaries without masking, so context beyond 1024 consists largely of *other documents*’ text, and the penalty compounds with the amount of extension. The corollary for Config A is what matters: since added context is worthless to harmful here by every within-method reading, Config A’s gains are not a context effect. On packed web text, the benefit of token averaging is data throughput per FLOP.

The experiment also answers the context-for-context question: when extended raw context is the goal, averaging obtains it more cheaply than widening the attention window. The two 2048-context arms finish at the same loss (4.6503 for $k=2$ averaging, 4.6505 for full attention), with the averaged route spending $1.26\times$ less training compute and holding half the KV cache and attention cost at inference for the same raw context. [TODO: report the full-position endpoints for the two 2048-context arms here, which is the comparable statement of this equality] The question matters most on data where long context actually helps (long documents, code), which we flag as follow-up work. [TODO: loss-vs-position curves; document-length distribution of FineWeb; PG19/code long-context experiment]

5.4 Pooling ablations: a learned content-plus-position rule wins at both compression ratios

Everything the transformer learns passes through the pooling function, so we ablate it at 50M under Config A with a fully matched protocol: identical architecture, data, batch size, update schedule, warmup, embedding tying, and evaluation, with fixed token budgets within each k (2.00B raw tokens at $k=2$; 4.07B at $k=4$). All runs within each k block share the same k and (except for the word-aligned variant) the same window scheme, so their native losses are mutually comparable. Figure 5 shows all trajectories; Table 5 the endpoints and quarter-budget checkpoints.

One comparability boundary applies: the word-aligned variant predicts a different position distribution (word starts only) at a slightly different realized compression, so its native loss is not measured on the same prediction task as the fixed-window variants; we report it as indicative and

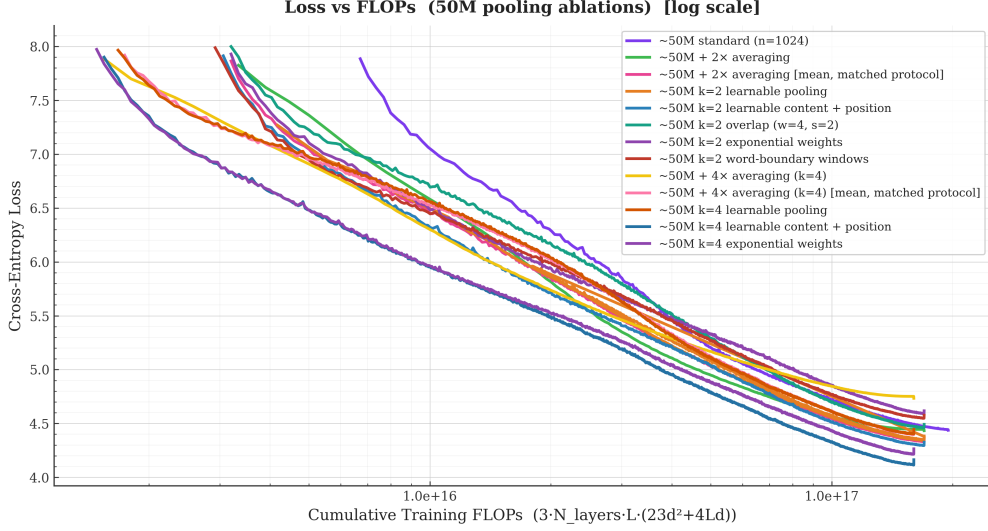


Figure 5: Pooling ablations at 50M: native eval loss vs. recomputed training FLOPs ($\log x$), all $k=2$ and $k=4$ variants overlaid with the $k=1$ baseline and the original mean-pooling runs. Curves are mutually comparable only within a k block and matched protocol; see text.

exclude it from the strict ordering (Appendix C).

Table 5: Pooling ablations at 50M (Config A), fully matched protocol. Native eval CE (nats/token) at the nearest logged checkpoint to each quarter of the token budget. Runs are comparable within a k block; the word-aligned row is measured on a different prediction task (Appendix C). [TODO: seed replicates and variance bars]

k	Pooling	25%	50%	75%	Final	Final PPL
2	learnable (content + position)	5.170	4.619	4.389	4.326	75.7
2	mean	5.174	4.649	4.425	4.363	78.5
2	learnable (content only)	5.222	4.675	4.443	4.383	80.1
2	overlap ($w=4, s=2$)	5.460	4.820	4.562	4.492	89.3
2	word-aligned (task differs)	5.413	4.872	4.640	4.577	97.2
2	exponential (fixed)	5.439	4.962	4.707	4.617	101.2
4	learnable (content + position)	4.951	4.463	4.224	4.163	64.3
4	exponential (fixed)	5.041	4.569	4.330	4.265	71.1
4	learnable (content only)	5.315	4.753	4.512	4.445	85.2
4	mean	5.326	4.756	4.505	4.446	85.2

One rule wins both regimes. The content-plus-position learned pooler is the best rule at both compression ratios, and it is the only rule that is: every alternative is regime-specific. At $k=2$ it finishes 0.037 nats ahead of mean pooling and is ahead at every quarter-budget checkpoint; at $k=4$ its margin over mean grows to 0.283 nats (0.102 over the next-best rule, the fixed exponential), again leading at every checkpoint. The pooler costs $d_{\text{model}} + 1 + k$ parameters, a negligible addition at any scale.

The sweep isolates why: position, not content, is what mean pooling misses. The remaining rows decompose the win. At $k=4$, a *fixed* recency weighting alone recovers most of the gap to the winner (4.265 vs. 4.446 for mean), while the content-only learned scorer recovers none of it (4.445, indistinguishable from mean). The explanation is architectural: the content scorer evaluates each embedding *in isolation, before any positional signal exists in the network*, so it cannot express “the last token of the window matters most”, which is exactly the prior that matters when four tokens collapse into one vector and the window’s final token is the one adjacent to the prediction target. At $k=2$ the same recency prior, applied as an aggressive fixed rule, actively hurts (4.617, the worst rule tested): with only two tokens per window, both carry useful signal, and systematically down-weighting the first discards it. The content-plus-position pooler resolves the tension by *learning* the positional profile per compression ratio, and content information then adds a further increment on top at both k . Of the structural alternatives, overlapping windows, which remove hard boundary cuts entirely, and word-aligned grouping both land worse than plain mean, so neither boundary effects nor linguistically meaningful grouping is the dominant mechanism.

Protocol sensitivity. The matched design of this sweep is not pedantry. Mean pooling trained under an earlier protocol ($\sim 8\times$ fewer optimizer updates for the same token budget, correspondingly different warmup exposure) lands 0.070 nats worse at $k=2$ and 0.288 nats worse at $k=4$ than the same model under the matched protocol: the update schedule alone moves endpoints by more than most method gaps in Table 5. This measurement is what motivates the update-count control on the main comparison (Section 6). [TODO: seed replicates; offset-ensemble rescoring of the pooling winners; learned weight profiles (Appendix D)]

6 Discussion and Limitations

Scale. The efficiency multipliers reported here come from the 50M–250M range; they do not by themselves establish a scaling law, which is why the 500M/1B arm of the study exists. Data logistics are part of this: at $k=2$ the averaged arm consumes twice the raw tokens, and the 250M $k=2$ run already exhausts FineWeb sample-10BT, so larger runs move to a $\geq 100\text{BT}$ slice. That introduces a data-distribution change between scales that we handle by fitting per slice. [TODO: 500M/1B results; decide slice handling once those runs are configured]

Optimizer-step control. Iso-FLOP comparisons at $k\times$ tokens give the averaged arm more optimizer updates at fixed batch size, and the protocol-sensitivity measurement of Section 5.4 shows that update count moves endpoints at these scales. We therefore run an explicit control: a $k=1$ baseline with halved batch size and doubled steps, matching the $k=2$ arm’s update count and schedule length at identical FLOPs. [TODO: add update-count control result]

Tokenizer-granularity control. Averaging $k=2$ tokens resembles, at a squint, simply using a tokenizer with $2\times$ longer average tokens. The two are not the same, since averaging preserves the fine vocabulary and composes embeddings dynamically, but a coarser-tokenizer baseline compared in bits-per-byte is the direct control. [TODO: coarser-tokenizer baseline in bits-per-byte]

Generation and downstream evaluation. Everything reported here is language-modeling loss. Rotating-offset decoding (Appendix B) provides token-by-token generation at amortized baseline cost, and downstream zero-shot quality (LAMBADA, HellaSwag, PIQA, ARC-E) is the natural

complement to the loss results. A small local decoder that emits all k tokens per window (Block-Transformer-style) is a further extension that would give native generation. [TODO: rotating-offset decoding benchmark; downstream evals]

Data dependence of the context result. The finding that extended raw context hurts is a statement about packed FineWeb at these scales, where most documents are shorter than 1024 tokens and packing crosses boundaries without masking. It is not a claim about long-document corpora, code, or masked packing, where the context-for-context advantage of averaging over full attention could translate into a genuine long-context win. We consider this the most promising unexplored branch of the work.

7 Conclusion

[TODO: Write after the 250M/500M/1B endpoints and the remaining full-position numbers land. Skeleton: (i) token averaging turns cheap data throughput into lower loss at fixed compute: 3.4% perplexity at $0.91\times$ FLOPs on the strictly comparable metric at 125M, equal-loss compute savings of $1.58\times$ (50M) and $1.44\times$ (125M); (ii) random-offset training and offset-ensemble evaluation resolve the comparability problem and make patch-style operation viable as a final operating point, with rotating-offset decoding for generation; (iii) extended raw context hurts on packed web text, so the gain is data-per-FLOP, while averaging is the cheaper route to context when context is wanted; (iv) a content-plus-position learned pooler is the best pooling rule at both compression ratios, and the positional prior is the ingredient mean pooling lacks. Close with the scaling question as the decisive open item.]

References

[TODO: complete bibliography when Related Work is written]

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., & Sifre, L. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.

A FLOPs Derivation

Per layer and per transformer position, counting multiply-accumulates as 2 FLOPs, forward pass:

- Attention projections: Q, K, V , and output, i.e. four $d \times d$ matmuls: $4 \times 2d^2 = 8d^2$.
- SwiGLU FFN with `ff_multiplier 2.5`: the combined implementation projects $d \rightarrow 5d$ for up and gate ($2 \cdot 5d^2 = 10d^2$), then projects down $2.5d \rightarrow d$ ($2 \times 2.5d^2 = 5d^2$): total $15d^2$. (A standard $4\times$ FFN would give $16d^2$; our architecture’s SwiGLU at 2.5 gives $15d^2$, and we use the exact value.)
- Attention scores and value aggregation: $2 \cdot 2Ld = 4Ld$ per position at sequence length L (causal masking halves the effective term; we follow the standard convention of quoting the full-matrix count and apply it uniformly to all arms, so ratios are unaffected). [TODO: state the causal-mask convention once and verify consistency with plotting code]

Total forward: $23d^2 + 4Ld$ per layer-position. Training multiplies by 3 (forward + backward). A sequence of `seq_len` raw tokens occupies $L = \text{seq_len}/k$ positions, giving Eq. (2). We recompute FLOPs from `tokens_seen` via Eq. (2) in every plot and table of this paper, including the preliminary 250M endpoints; the `cumulative_flops` column logged by early training code was inconsistent across runs and is never used.

Embedding and output-head FLOPs are excluded from Eq. (2) on both arms; at these scales the head contributes a few percent and equally to all arms of a comparison. [TODO: quantify head share at 50M for completeness]

B Offset-Ensemble Evaluation: Details

For a k -averaged model and evaluation sequence $t_{1..T}$, offset $o \in \{0, \dots, k-1\}$ discards the first o tokens, forms $n_o = \lfloor (T - o)/k \rfloor$ windows, embeds and pools them, and runs the transformer on the first $n_o - 1$ compressed positions. The prediction at compressed position j (1-indexed) is scored against the raw token at position $o + jk + 1$. Across offsets, the predicted position sets are disjoint and their union is every position $> k$ (up to edge effects at the sequence tail), each predicted exactly once from its full compressed prefix. Mean NLL is computed by pooling all scored predictions across offsets and sequences. For the restricted baseline, the $k=1$ model’s per-position NLLs are gathered on exactly this position set. Total cost is k forward passes at length $\approx T/k$, i.e. approximately one baseline pass at length T (the attention term is actually cheaper by a factor k).

Generation rotates the anchor: to emit the token at raw position $p+1$ given a compressed cache for some offset, the decoder uses the offset $o \equiv p \bmod k$ whose window grid ends exactly at p . Maintaining k caches (one per residue class) amortizes to approximately one baseline decoder’s cost, since each cache is $1/k$ the length. [TODO: benchmark rotating-offset decoding; measure downstream tasks]

C Word-Aligned Windows: Algorithm and Caveats

A token begins a word iff its string form starts with the BPE whitespace marker (a leading `␣` in GPT-NeoX byte-level BPE) or a newline marker, or is a special token; a boolean lookup table over the padded vocabulary is built once from the tokenizer. Grouping is greedy per sequence: accumulate tokens into the current window; close the window when it holds $\geq k$ tokens *and* the next token starts a new word, or unconditionally at $2k$ tokens (so a pathologically long word cannot grow a window without bound); drop a trailing partial window. Windows therefore span 2–4 tokens at $k=2$, never end mid-word (unless a word exceeds $2k$ tokens), and realized compression on FineWeb is $\sim 2.1\times$ at $k=2$. Batch handling truncates all sequences in a batch to the minimum window count; the grouping loop costs $\sim 1\text{ms}$ per batch at $B=16$, $T=1024$ (negligible). Targets are the first token of the next window, which for this variant is always a word-start token.

Two comparability caveats follow. First, realized compression exceeds the nominal $2\times$, so iso-FLOP accounting must use the realized ratio (the transformer sees $\sim 5\%$ fewer positions per raw token than uniform $k=2$). Second, the predicted-position distribution differs (word starts only), so its native eval loss is not measured on the same prediction task as the uniform variants; the offset-ensemble protocol does not directly apply to variable-size windows either. A bits-per-byte comparison, or a fixed-position rescoring, is required before strong claims; we present the word-aligned row as indicative only. [TODO: bits-per-byte rescoring for the word variant]

D Pooling Modules

Content-based learnable pooler. Shared scorer $s_i = w^\top e_i + b$ with $w \in \mathbb{R}^{d_{\text{model}}}$, $b \in \mathbb{R}$, zero-initialized so that $\alpha_i = 1/k$ exactly at initialization (training starts at mean pooling); $\alpha = \text{softmax}(s_{1..k})$ within each window; $\bar{e} = \sum \alpha_i e_i$. Parameters: $d_{\text{model}} + 1 = 513$. Trained jointly with the LM by the ordinary objective; no auxiliary reconstruction loss is used in these experiments.

Position-aware learnable pooler. $s_i = w^\top e_i + b + p_i$ with $p \in \mathbb{R}^k$ a learned per-slot bias. Initialization: $p = 0$ (uniform) at $k=2$, where the recency prior hurt; $p_i \propto$ the exponential profile with last-to-first ratio 20 at $k=4$, where it won. Parameters: $d_{\text{model}} + 1 + k$. Both terms train end-to-end; inspecting the learned α_i by position and token type is part of the planned analysis. [TODO: report learned weight profiles]

Fixed weighting schemes. $w_i \propto \exp(\alpha i)$ with $\alpha = \ln(20)/(k-1)$ (exponential; the trained representative), plus linear, Gaussian, and triangular alternatives implemented but not trained at scale. All schemes are L_1 -normalized.

E Full Run Table

Table 6: All runs referenced in this paper, with canonical result files. “Protocol” distinguishes the matched protocol used for all canonical runs (16,384 raw tokens per update, tied embeddings, 2k-step warmup) from the original protocol of earlier runs, which are retained as references only. All models use tied embeddings. [TODO: add exact batch size, world size, and step counts per run; add seeds; add 250M/500M/1B rows as they complete.]

Run	k	Raw tokens	Final eval	Protocol	Notes
50M $k=1$ baseline	1	1.0B	4.437	matched	canonical baseline
50M $k=2$ mean	2	2.0B	4.363	matched	canonical
50M $k=4$ mean	4	4.07B	4.446	matched	canonical
50M $k=1$ update-count control	1	1.0B	[TODO:]	matched	half batch, $2\times$ steps
50M $k=1$, 2048 full attn	1	1.0B	4.651	original	context control
50M $k=2$, seq 2048 (Config B)	2	2.0B	4.650	original	context arm
50M $k=4$, seq 4096 (Config B)	4	4.07B	5.174	original	context arm
125M $k=1$	1	2.5B	4.204	original	
125M $k=2$	2	5.0B	4.171	original	
250M $k=1$	1	4.18B/5B	3.458	original	incomplete
250M $k=2$	2	8.36B/10B	3.438	original	incomplete
50M $k=2$ learnable+pos	2	2.0B	4.326	matched	pooling sweep
50M $k=2$ learnable (content)	2	2.0B	4.383	matched	pooling sweep
50M $k=2$ overlap $w4s2$	2	2.0B	4.492	matched	pooling sweep
50M $k=2$ word-aligned	2	2.0B	4.577	matched	metric caveats
50M $k=2$ exponential	2	2.0B	4.617	matched	pooling sweep
50M $k=4$ learnable+pos	4	4.07B	4.163	matched	pooling sweep
50M $k=4$ exponential	4	4.07B	4.265	matched	pooling sweep
50M $k=4$ learnable (content)	4	4.07B	4.445	matched	pooling sweep
50M $k=2$ mean (old protocol)	2	2.0B	4.433	original	reference only
50M $k=4$ mean (old protocol)	4	4.07B	4.734	original	reference only

F Training Instabilities and Excluded Runs

For completeness and reproducibility we document the detours.

1. **RoPE bug.** An early implementation bug in the rotary embedding invalidated the first tied-embedding run and several early k -sweep results. All affected logs are quarantined (`*_rope_bug` suffixes in the results tree) and no number in this draft derives from them.
2. **50M baseline instability.** The original 50M $k=1$ run suffered an early loss explosion and converged ~ 1 nat too high; a clean rerun (the 4.437 baseline used throughout) matches the cross-scale pattern. All runs at this scale show a large transient loss spike in the first $\sim 2k$ steps that self-recovers; it is an initialization artifact, clipped in plots.
3. **The seq_len accident.** The first $k=2$ runs were launched with the baseline’s raw `seq_len` rather than the doubled value originally intended. The accidental setting holds raw context fixed and isolates the data-per-FLOP effect, and it became the paper’s primary configuration (Config A); the originally intended setting became the Config B ablation.
4. **Duplicated ablation log.** The $k=2$ learnable run’s CSV contains two interleaved logging streams and one malformed line (from a resumed job double-writing); the two recorded endpoints agree to 0.004 nats (4.379 vs. 4.383) and the file is cleaned deterministically before plotting.

G Hyperparameters

Table 7: Shared hyperparameters. [TODO: fill exact per-run batch sizes and world sizes from run logs.]

Optimizer	AdamW
LR schedule	cosine, decay to 10% of peak over total steps
Warmup	2,000 steps
Peak LR	$2.0/1.6/1.4 \times 10^{-4}$ (50M/125M/250M)
Precision	bfloat16 autocast
Raw <code>seq_len</code>	1024 (Config A; Config B as stated)
Tokenizer	EleutherAI Pythia-70m (GPT-NeoX BPE), vocab 50,257
Vocabulary padding	50,304
FFN	SwiGLU, expansion 2.5
Positional encoding	RoPE (applied to compressed positions)
Chinchilla reference constants	$A=406.4$, $\alpha=0.3392$, $B=410.7$, $\beta=0.2849$, $E=1.6934$